

EE272 Homework 5

HLS Design Optimization

Due: Feb 12, 2026, 11:59 pm

Released: Feb 5, 2026

Now that you have a functional HLS design, the goal of this homework is to improve its performance (reduce the number of execution cycles). In [section 1](#) to [3](#) we present a few fundamental HLS optimization techniques. The most effective ways to improve performance in HLS are pipelining and unrolling; however to implement these successfully you need to guide the HLS tool by inserting pragmas in your code. In [section 4](#), you will capture the performance numbers for your design point and potentially fix your design under the guidance we provide. In [section 5](#), we want you to analyze the performance difference between your HLS accelerator and the Verilog accelerator.

You should work with a 16×16 systolic array, and must keep the default loop tilings we have provided below. You are allowed to change I/O bandwidths and buffer sizes, but the sum should not be larger than the default values.

Here are the default tiling values we will use in this homework:

Table 1: Default tiling for ResNet-18 layers.

layer	OY1	OY0	OX1	OX0	OC1	OC0	IC1	IC0	FX	FY	Stride
conv1	8	14	8	14	4	16	1	3	7	7	2
conv2_x	4	14	4	14	4	16	4	16	3	3	1
conv3_1	4	7	4	7	8	16	4	16	3	3	2
conv3_x	4	7	4	7	8	16	8	16	3	3	1
conv4_1	2	7	2	7	16	16	8	16	3	3	2
conv4_x	2	7	2	7	16	16	16	16	3	3	1
conv5_1	1	7	1	7	32	16	16	16	3	3	2
conv5_x	1	7	1	7	32	16	32	16	3	3	1

Table 2: Default memory depths.

Input Buffer Depth	4096
Weight Buffer Depth	8192
Accumulation Buffer Depth	256

Table 3: Default I/O bandwidths.

	Precision (Bits)	Number of Pixels	Total Width (Bits)
Input	8	4	32
Weight	8	4	32
Output	32	1	32

1 Memory Optimization

In homework 4 handout, we briefly mentioned that you will need to set the proper `WORD_WIDTH`, `STAGE_REPLICATION`, `INTERLEAVE` and `BLOCK_SIZE` for memories in order to synthesize them into the expected hardware. In fact, your code can still synthesize without them, but the resultant performance might not be what you expected.

We have discussed the purpose and effect of each of these settings in the lecture. Please review the lecture slides, as well as the Catapult User Reference to figure out the proper values for them.

2 Loop Pipelining and Unrolling

As we discussed in class, to add pipelining and unrolling to your design you will use the pragmas `#pragma hls_pipeline_init_interval` and `#pragma hls_unroll`, respectively. The HLS lecture delves deeply into how pipelining and unrolling work, so please review it before starting. For unrolling, think about what hardware you want unrolled spatially in your design. Make sure to place `#pragma hls_unroll yes` before these `for` loops. For pipeline initiation intervals, consider where you would need to overlap the execution of `for` loops for higher throughput. For loops you would like to run with an initiation interval of one, you can use `#pragma hls_pipeline_init_interval 1`. Below are two examples of unrolling and pipelining. In the first example, we will generate N adders in hardware to perform the entire loop in parallel. This is a good candidate for unrolling because there are no data dependencies. In the second example, we can use an initiation interval of 1. This means that instead of taking 9 cycles to complete, it will take 5 cycles as shown in [Figure 1](#). This loop would not work with unrolling due to data dependencies, and is a solid candidate for pipelining since there is no redundant hardware.

Example 1

```
#pragma hls_unroll yes
loop_1: for(int i = 0; i < N; i++) {
    a[i] = b[i] + c[i];
}
```

Example 2

```
#pragma hls_pipeline_init_interval 1
loop_1: for(int i = 0; i < 3; i++) {
    c = a + b; // OP1
    d = a * c; // OP2
    e = a - d; // OP3
}
```

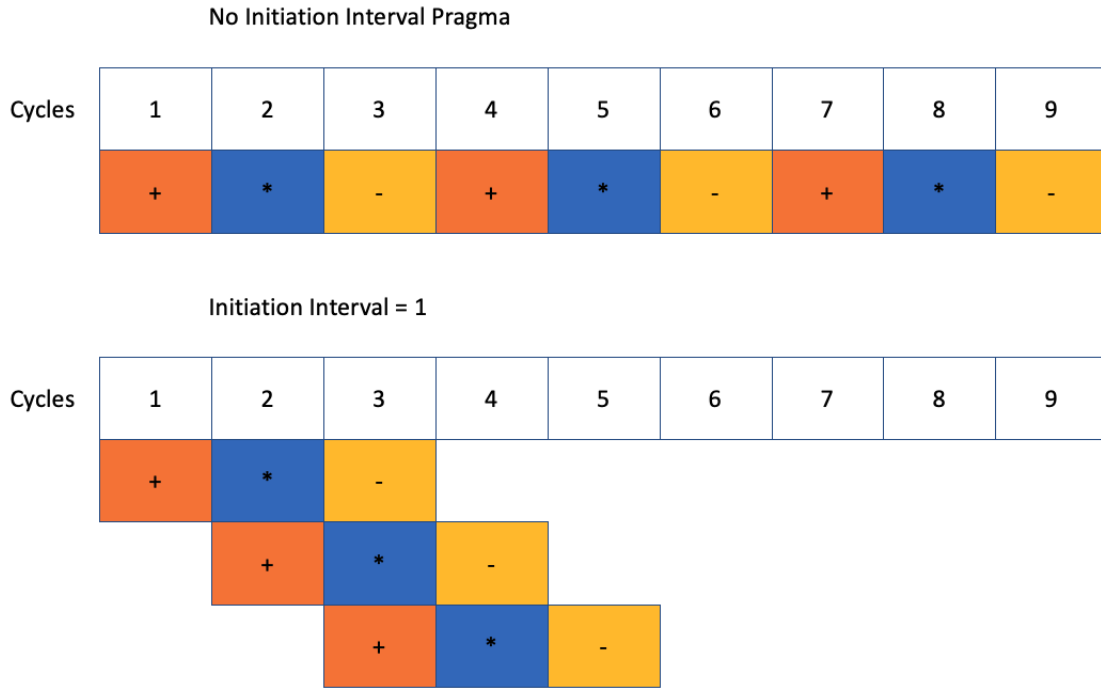


Figure 1: Higher throughput with initiation interval 1.

Pipelining and unrolling will likely be the most productive sources of optimization that you find in this assignment, so make sure that you understand both concepts very well. If your runtime is much larger than your Verilog implementation, it is likely because you are not pipelining or unrolling enough. We recommend that you carefully look at every single loop in your design and consider what the hardware would look like and if you could improve performance by pipelining or unrolling it.

2.1 Resolving Scheduling Failures

Although loop pipelining and unrolling are good strategies for improving performance, they may create scheduling failures as mentioned in class. Two most commonly seen failures are resource competition failure and memory dependency failure.

Resource competition occurs when Catapult tries to schedule multiple operations in the same cycle and those operations are fighting for limited hardware resources. For example, if

multiple read operations are scheduled into the same cycle after pipelining or unrolling, and the memory is mapped to a single-port SRAM, then resource competition occurs because such a memory can only serve one read per cycle. To solve this problem, you need to either increase the hardware resources or re-schedule the operations. For instance, in this example you can map the memory to a two-port SRAM or increase the word width to serve higher bandwidth. You can also manipulate the pipelining and unrolling to avoid such conflicts.

Memory dependency becomes an issue when you try to read from and write to the same memory in the same cycle. If Catapult cannot prove that the read address and write address are different, memory dependency failure occurs to prevent potential future errors. However, if you know that the read and write addresses are not the same, you can mark this error as a false alarm by setting the `ignore_memory_precedences` constraint in the Tcl files:

```
ignore_memory_precedences -from <start_point> -to <end_point>
```

`start_point` and `end_point` can be found in your error report. You might also find it helpful to use the [Tcl for-loop](#) when you are setting these constraints because there might be multiple memory dependencies you need to ignore.

3 Optimizing Loop Bounds

There are quite a few loops in your design that have conditional bounds. In Verilog, these conditional loops would be typically implemented as finite state machines where you precisely control the bitwidth of the loop variables and the exact cycles when the loop bound conditions are met. In HLS, you instead describe these loops in C++. However, the way that these loops are written can have a dramatic impact on the resulting latency and throughput of your design.

Take, for example, the following loop:

```
void loop_cond_accum(int din[4], int &dout, int ctrl) {
    int acc=0;
    ACCUM:for (int i=0; i<ctrl; i++) {
        acc += din[i];
    }
    dout = acc;
}
```

While this is a perfectly reasonable looking loop in C++, there are a few things to consider when targeting HLS. First, using a variable as a loop bound often requires additional hardware, using a variable as a loop upper bound requires an extra cycle to test the stop condition, and using an `int` for the loop bound results in unnecessarily large loop counters. Even if you know that the value of `ctrl` will never be above 4, the HLS tool is unaware.

[Figure 2](#) shows how this loop would be synthesized into hardware. Since `ctrl` is a 32 bit integer, we need a 32-bit counter and 33-bit loop exit condition.

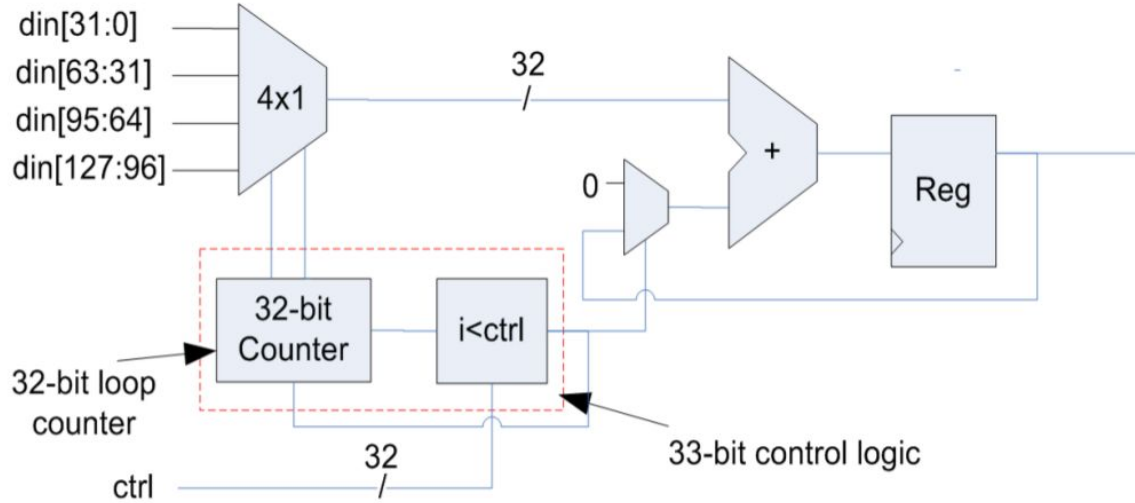


Figure 2: Hardware for conditional loop bound with `int` loop counters.

In order for the HLS tool to reduce the bit width of the loop counter, the upper bound of the loop should be set to a constant. However, this obviously changes the behavior of your circuit, so you need to add a loop exit condition inside your loop. The transformed loop would look like this:

```
#define MAX_CTRL_VAL 4
void loop_cond_accum(int din[4], int &dout, ac_int<3,false> ctrl) {
    int acc=0;
    ACCUM:for (int i=0; i<MAX_CTRL_VAL; i++) {
        acc += din[i];
        if (i == ctrl-1) {
            break;
        }
    }
    dout = acc;
}
```

In this example, we know that the maximum value that `ctrl` can take (`MAX_CTRL_VAL`) is 4, so our loop counter only needs to be 3 bits wide. The `break` statement preserves the functionality of our loop in every case except when `ctrl` is 0. If this is possible in our design, extra control logic is needed. Finally, we also change our conditional loop bound `ctrl` to an `ac_int` of size 3, reducing the bitwidth of our loop exit condition.

Figure 3 shows the resulting hardware for the optimized loop. We now only need a 3-bit counter and 3-bit control logic. In addition, since our loop exit condition now happens at the end of the loop instead of the beginning, we have removed a cycle from the loop execution time. For more information, see Sections 4.3.4 - 4.3.6 in the HLS Bluebook (/cad/mentor/2019.11/Catapult_Synthesis_10.4b-841621/Mgc_home/shared/pdffdocs/hls_bluebook.pdf).

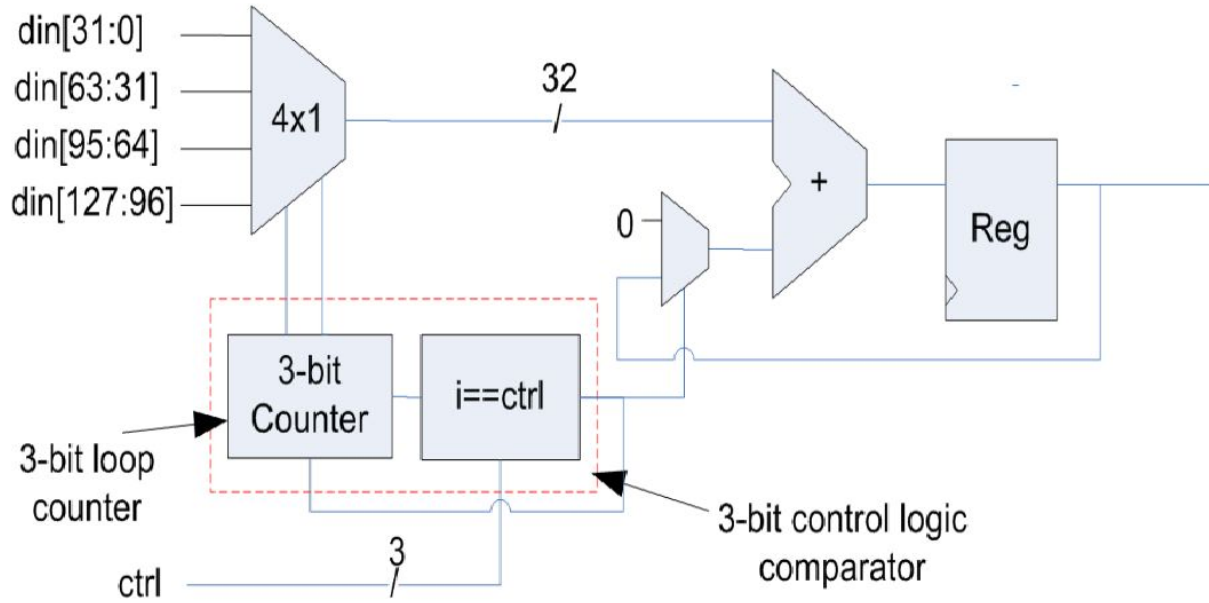


Figure 3: Hardware for optimized loop.

Hint: You may need to re-evaluate the loop pipelining in `InputDoubleBuffer.h` and `WeightDoubleBuffer.h`. The previous pragmas you placed may not work now.

4 Design Point Capture

Before proceeding, we want you to evaluate the performance of your current design after implementing the optimizations above. **In your report, please answer the following questions.**

Table 4: Ideal and analytical performance for homework 3 Verilog implementation.

Conv	Ideal	Analytical Numbers with I/O Constraints		Unoptimized Verilog Simulation	
	Cycles	Cycles	MAC Utilization	Cycles	MAC Utilization
1	460992	2474767	0.1875	2475377	0.186
2_x	451584	451584	1.0	461165	0.979
3_1	225792	315648	0.715	320333	0.705
3_x	451584	589824	0.766	595517	0.758
4_1	225792	305280	0.740	313437	0.720
4_x	451584	589824	0.766	599997	0.753
5_1	225792	300096	0.752	315389	0.716
5_x	451584	589824	0.766	609149	0.741

1. Find the number of cycles your current HLS design takes to run each ResNet-18 layer. The default clock period is 5 ns, so divide the simulation time by 5 to find the cycles.
2. Find the MAC utilization for each layer by calculating the ratio between ideal cycles, listed in [Table 4](#), and your simulation cycles above. We have provided you the numbers from our code at the same design point in [Table 5](#). If your numbers are very far away from the “After Optimization” numbers, please carefully read [subsection 4.1](#).
3. In your report, please attach a waveform of your HLS simulation with signals that show the correct double buffer behavior. If you had to go through the double buffer fix described in the next section, please also attach the waveform before your fix (it will help you understand deeper what went wrong!)

Hints on how to identify the correct double buffer behavior are in the following section.

4.1 Double Buffer

In lectures, we presented how to properly write HLS code so that a double buffer is inferred. If your performance is very far away from the reference numbers we provided, it is likely that your coding style did not correctly infer a double buffer. In this section we will give you some guidance to fix your double buffer code.

First, please carefully review HLS Bluebook Section 8.1.7 (it is short). **Note:** From our experiments, we have seen two different coding styles with which the double buffer is synthesized into the correct hardware. Here we present one method that we think follows the instructions in the Bluebook more closely. Even if you have hit the performance goal with your current code, you may have just been lucky to write a working code in the first shot. So, please still carefully read through this section.

To identify whether or not your double buffer indeed behaves as expected, here are a few things you can check:

1. Look at the NCSim waveform of your HLS design and compare to the Verilog waveform ([Figure 4](#)), do you see a similar double buffer read/write behavior?

In [Figure 4](#), the first marker marks the end of initial loading of inputs and weights, and the following markers mark when each tile has finished processing, i.e. when banks are switched and new weights start to load. In your HLS simulation, you should see similar boundaries by looking at `input_serial_rsc_dat`, `weight_serial_rsc_dat` and `output_serial_rsc_dat` signals. Otherwise, double buffer is not correctly inferred for you.

2. If your code is written correctly, the local temporary memory struct `tmp` (or whatever you named it) should be optimized away. After synthesis, in the Catapult scheduler, review whether there is any memory read/write associated with `tmp`. You should not see any operations like [Figure 5](#) in the scheduler. You also should not see any signal with `tmp` in SimVision Design Browser when you run simulation.

Now, if you identified that your double buffer behavior is not the same as the Verilog counterpart, you are going to fix the problem.

Two key points from the Bluebook that we want to highlight:

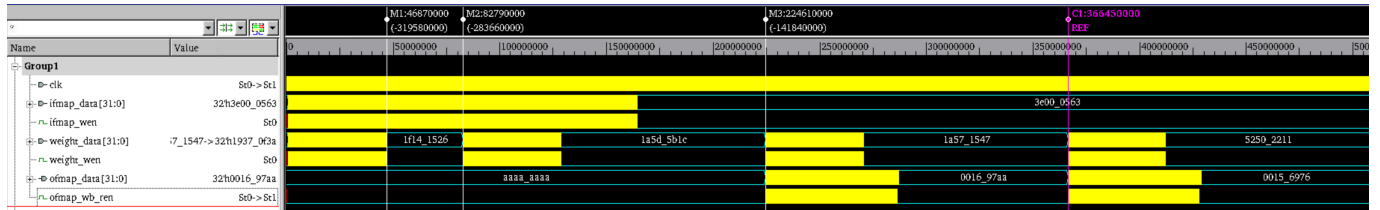


Figure 4: Verilog conv2_x simulation waveform.

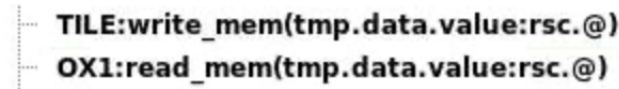


Figure 5: Memory operations on `tmp` local memory struct.

- Do not write to the temporary buffer after you have written the temporary buffer to the `ac_channel`.
- It's also important to limit the scope or lifetime of the local struct to avoid inferring extra memories. Both the declaration of the temporary buffer and its read from or write to an `ac_channel` must be in the same scope.

In order to achieve this, here are a few concepts you need to understand first:

1. Hierarchical blocks are created for functions on which you specify the pragma `#pragma hls_design interface` (i.e. the `run` functions). Once the HLS code is synthesized into RTL, those interface functions will be executed repeatedly as long as there is data available in the channels. So, the `run` function will run multiple times, despite the fact that in the C code it is not called multiple times. This is why in `SystolicArrayLooper` you need to write to `paramsOut` and `loopIndicesOut` for every iteration of the outer loops.

However, C `ac_channels` are non-blocking, so each interface function can be called when there is not enough data in the `ac_channels`. That is the reason for having:

```
#ifndef __SYNTHESIS__
while(paramsIn.available(1))
#endif
```

The C simulation will go on as long as there is still data available in the input channels. You can see this is turned off in synthesis.

2. In RTL simulation, whenever there is data in one channel of the interface, it effectively kicks off the `run` function again, which means the other channels should also have data available to be read, otherwise you will run into an empty channel error.

Consider the following example:


```
void run (ac_channel<int> &a, ac_channel<int> &b) {
    a.read();
    for (int i = 0; i < 3; ++i) {
        b.read();
    }
}
```

For each execution of the function, channel **a** is read once, and channel **b** is read three times. If channel **a** is written 1 time and channel **b** is written 6 times, after the first execution channel **a**'s size will be 0 and channel **b**'s size will be 3. The second execution will be kicked off due to data available in channel **b**, but it will cause error because channel **a** is empty and cannot be read.

This is why in `SystolicArrayLooper` we had to write to `paramsOut` whenever we write to `loopIndicesOut`. But due to the freedom of writing C++ in various styles, this can potentially not be caught in C simulation, allowing C tests to pass while RTL tests fail. One way to make sure the results are consistent in C tests and RTL tests is to put the entire body of the function in the `while(a.available(1))` loop. This way the C test will fail if the channel sizes are not aligned properly. You can choose to check any channel since they are expected to reduce to 0 at the same time. Remember to put it inside the `#ifndef __SYNTHESIS__` guard.

```
void run (ac_channel<int> &a, ac_channel<int> &b) {
    #ifndef __SYNTHESIS__
    while(a.available(1)) {
        #endif
        a.read();
        for (int i = 0; i < 3; ++i) {
            b.read();
        }
        #ifndef __SYNTHESIS__
    }
    #endif
}
```

Once you understand these two concepts, you should be ready to fix your double buffer. You will need to add a similar `while` loop to your Reader and Writer. You will have to make sure `paramsIn` is written for the correct number of times in order for all inputs and weights (coming from the testbench) to be written into memory. An example of that can be found in `ParamsDeserializer` and `Serializer`.

Hint: You may need to re-evaluate the loop pipelining in `InputDoubleBuffer.h` and `WeightDoubleBuffer.h`. The previous pragmas you placed may not work now.

Table 5: Reference performance for ResNet-18 layers after each optimization.

	After Optimization		After Inner Loop Optimization	
Conv	Cycles	MAC Utilization	Cycles	MAC Utilization
1	2855006	0.161	2474079	0.186
2_x	530266	0.852	460827	0.980
3_1	373050	0.605	298745	0.756
3_x	742698	0.608	594701	0.759
4_1	376650	0.599	301009	0.750
4_x	747306	0.604	597973	0.755
5_1	383850	0.588	305921	0.738
5_x	756522	0.597	604901	0.747

5 Inner Loop Optimization

After applying the optimization techniques and double buffer fix, you should be able to hit the target performance in [Table 6](#). If not, please double-check the following items:

- Whether memory settings are correct. Word width and interleave/block size will impact the memory access speed.
- Whether pipelining and unrolling have been applied to the maximum capacity.
- Whether there is any conditional loop bound left.

You may have noticed that, even after applying the optimization techniques, your HLS design performance is still very far behind the Verilog version. This is because of the way the inner loop is written (the for loop labelled as `INNER_LOOP` in `SystolicArrayCore.h` in the solution code). Study the inner loop in the HLS code and the homework 3 Verilog `conv_controller.v` closely and **answer the following questions**.

1. Given a 16x16 systolic array, calculate the ideal cycles needed to complete one tile of outputs for all layers for both the Verilog and HLS design. Use the default tiling we provided in [Table 1](#).
2. Compare the two sets of numbers you found above, and explain where the extra cycles in the HLS version come from.
3. For the HLS version, determine the performance bound for each layer analytically.

Hint: You have done this in homework 3 and know what is the performance bound for each layer. The ideal cycles required to read/write a tile for all buffers are the same as before, since you have completed the double buffer fix. The only difference will be the ideal cycles for MAC computations.

In order to match the Verilog performance, you will need to change the code structure so that the inner loop in the `SystolicArrayCore` covers all IC1, FY, FX, OX0 and OY0 loop nests. This will introduce a fair amount of complexity to the `SystolicArrayCore` logic, but you will exceed the Verilog performance once it is implemented. See [Table 5](#) for the performance numbers from our code.

6 Compiling HLS

For this homework, you will use the same commands to compile and run your code as the last homework.

- To generate the RTL design run:

```
$ make
```

The file `dnn-accelerator-hls-unoptimized/build/Conv.v1/concat_rtl.v` will contain the generated RTL design. You can get to the top level module by searching for module `Conv`, and then look inside it hierarchically.

- To run an C test, use the command:

```
$ make c_test
```

- To run an RTL test, use the command:

```
$ make rtl_test
```

- You will also be able to get the simulation time without opening the GUI:

```
$ make rtl_test_no_gui
```

- When NCSim launches, several signals are added to the waveform window by default. Input and output ports for all the blocks are added by default, as well as several signals that indicate when the blocks in your design are active.

Press the run button on the top to start the simulation as shown in [Figure 6](#). The red `_error_cnt_sig`, which appears in the GUI after the top level ports, will tell you if there is a mismatch between the C++ code and the generated Verilog. Verify that all the outputs match.

- If you make changes to your code and recompile and see errors that you can not decipher, sometimes running `make clean` will help, before recompiling.

```
$ make clean
```

- When optimizing your HLS design, it will be helpful to look at the schedule for each block. After compiling your design and opening Catapult, click on the Schedule tab to open up the schedule for a block. Select between the different blocks in your design in the dropdown as shown in [Figure 7](#).
- **Check if the throughput of each block meets what is expected.** If the schedule is not correct, pipeline and unroll loops.

6.1 Autograder

The autograder can help run the commands required to test your design. In this assignment, you must test both the functionality of your design in C and in Verilog and make it performant.

To test in C, use the following command:

```
$ python3 autograder.py c_test -l layers/resnet_conv2_x_params.json
```

If you would like to run an RTL test, use the command:

```
$ python3 autograder.py rtl_test -l layers/resnet_conv3_1_params.json -v
```

If you want to debug the output of the commands that the autograder is running, use the verbose option `-v` or run the layer manually as indicated in the previous subsection.

Once your RTL is built, if you want to test a different layer without making any additional code changes, you can pass in `-n` to prevent rebuilding RTL. For example,

```
$ python3 autograder.py rtl_test -l layers/resnet_conv4_1_params.json -v
# And now you want to run another layer without rebuilding, you can do
$ python3 autograder.py rtl_test -l layers/resnet_conv4_x_params.json -v -n
```

Note that if you made any change to your source, you should not use this option.

If you do not pass in any layer json file, the autograder will use `small_layer1.json`. You can pass in as many layers at a time as you want, just leave a space in between the JSON file names. Also, if you would like to test all the ResNet-18 layers or small layers at once, use the commands:

```
$ python3 autograder.py c_test -l all
$ python3 autograder.py rtl_test -l all
```

The RTL test command will set up your RTL simulation with the specified layers and run the test. The RTL test has to compile your HLS code to RTL and run a lengthy simulation, so this command will take a while.

Note that if you get an error in your RTL test, a useful way to debug is to look at the waveform. Use the `make rtl_test` command to pull up simulation waveform viewer and refer to the previous section on running the simulation.

If you need any help with the commands for the autograder use:

```
$ python3 autograder.py --help
```

7 Evaluating Accelerator Performance

Your goal performance for each layer is in Table 6. If you reach this performance for each layer, you will receive full points. You will receive extra credit for hitting the unoptimized Verilog cycle numbers in Table 4 (our homework 3 solution).

In your report, please write:

1. What code optimizations did you make and how did they improve utilization or reduce the size of your design?
2. In a table, list the number of cycles your final optimized accelerator takes to run each layer and the MAC utilization.
3. Comparison of the HLS-generated accelerator with the hand-generated one from homework 3 in terms of performance.

8 Code Submission and Grading

Please submit your code as a **.tar.gz** file to Gradescope. To pack things up, use:

```
$ tar -czvf filename.tar.gz /path/to/your/directory
```

The grading of this assignment will be based on the correctness of your code, the performance you achieve, and your report. Please name your files using your and your partner's full names:

Table 6: Target performance in cycles.

Layer	Cycles	MAC Utilization
conv1	2910000	0.158
conv2_x	540000	0.836
conv3_1	380000	0.594
conv3_x	755000	0.598
conv4_1	384000	0.588
conv4_x	760000	0.594
conv5_1	391000	0.577
conv5_x	770000	0.586

- `firstname1_lastname1-firstname2_lastname2.tar.gz`
- `firstname1_lastname1-firstname2_lastname2.pdf`

9 Appendix: Characterization of Technology Libraries in Catapult

For this class we are using the SkyWater 130nm PDK (Process Design Kit). This is an open-source 130nm technology node that can be manufactured, so we are using it to allow students to tape out chips in EE372. The Catapult HLS tool uses information from this PDK to give the designer estimates of the timing and energy of their chip design.

At a high level, Catapult does a kind of synthesis where it translates your design into smaller building blocks ranging from individual logic gates to arithmetic structures like multipliers. This allows Catapult to give you an evaluation of your design without having to do full synthesis, which can take a long time.

To facilitate this, it uses a data structure called a library which contains the characteristics of each hardware building block. For example, one entry in this library is a `not` operation. The library entry contains the clock period, area, delay, and slack of this block at several different design points: fastest, smallest, and several points in between.

Each time you want to change the PDK you are targeting, you need to characterize the technology in Catapult using the Library Builder. This process is very straightforward, but it takes several days to complete, so we have completed this step for you. You can use this section as a reference in case you want to do library characterization in the future.

Catapult provides a user reference guide specifically for the Library Builder, it is located here:

/cad/mentor/2019.11/Catapult_Synthesis_10.4b-841621/Mgc_home/shared/pdfdocs/catapult_lb_useref.pdf

Catapult has provided a Library Wizard that walks you through how to generate the library. You will need the technology library file associated with your PDK in the Liberty format to start. Once you give the Library Wizard a path to this file along with a name for your new library, you will need to check just a few parameters before starting the characterization. Specifically, you will need to make sure that the library settings such as the operating conditions, time units, and capacitance units match the units in your Liberty file. You may also specify some I/O timing information like the external capacitive load or external driver cell library. Finally, you may change the default characterization settings as well. You can change the number of characterization points as well as the fastest and slowest clock period for characterization.

In our experience with SkyWater, the Library Wizard could not load the Liberty file, so we used the manual library creation tool instead. The manual tool has all of the same options as the library wizard, but does not hang when loading in the Liberty file. We also ran into an issue when trying to use Synopsys Design Compiler (DC) as the characterization tool. Using Synopsys DC gave us errors that indicated that the tool could not find the logfile section of the technology library. To get around this error, we used the Mentor Graphics Oasis-RTL synthesis tool instead, which gave us no issues.

After creating the new library, you must characterize all of the building blocks in the library. This is as simple as clicking the **Characterize** button and waiting for a few days. Once this has completed, you can save the library and load it into any of your HLS designs using the command: `solution library add library_name`. An example of this can be seen in file `scripts/generic_libraries.tcl` in the homework repository.

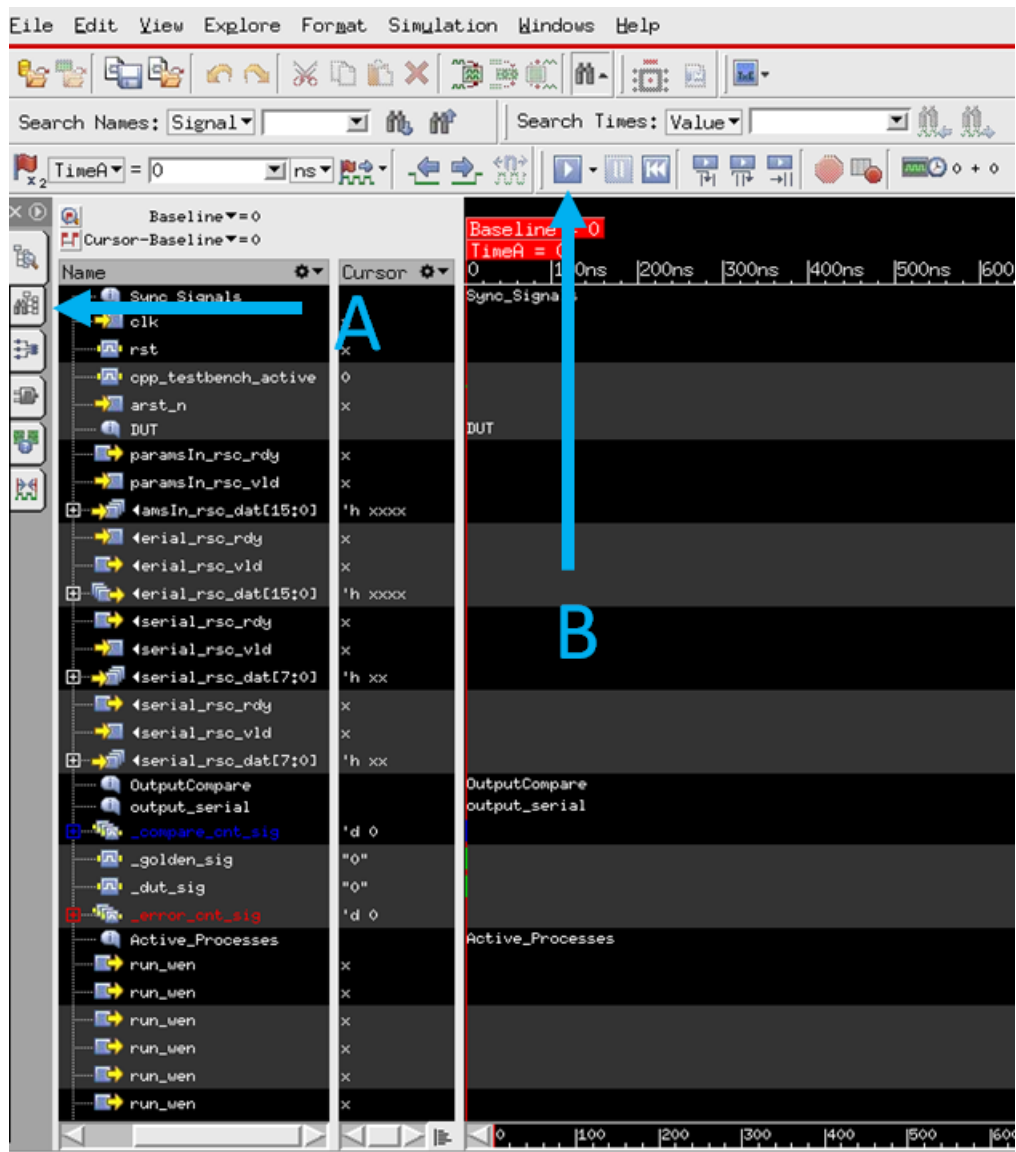


Figure 6: Click on the Hierarchy tab in (A) to go through the design hierarchy and find signals you want to add. Click the start button in (B) to run the simulation.

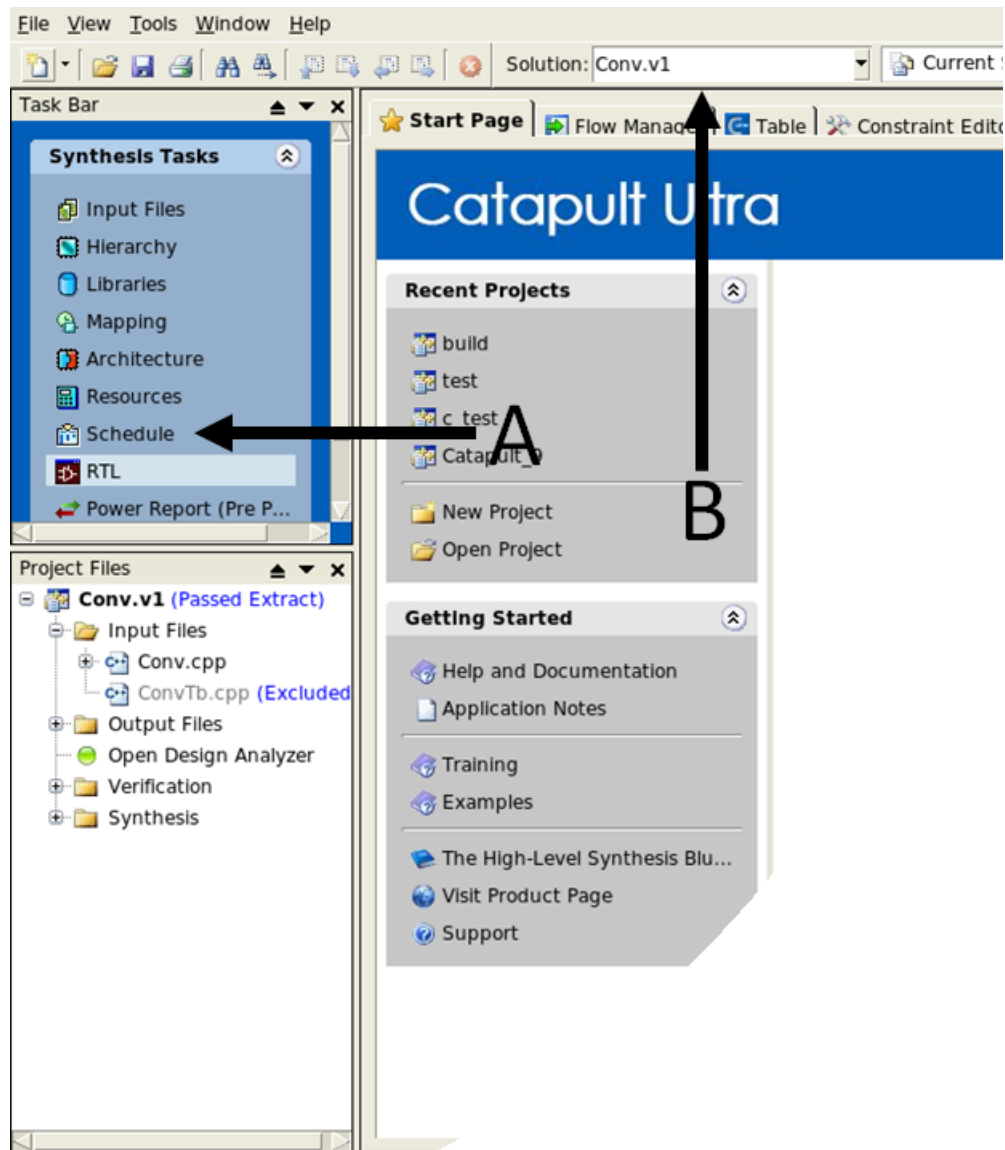


Figure 7: Click on (A) to switch to the Schedule step and click on the dropdown in (B) to switch between blocks.